

Canonical Methods to Reduce Metamers — Tetrachromacy + Ring/Spiral Buffer Pipeline

Executive Summary

This document describes a canonical collection of methods — chosen to be practical for a laptop-camera setup — that together reduce metamerism and minimize the nullspace of a sensing operation for video-based scene segmentation and shape discrimination. The focus is on engineering a tetrachromacy-like sensor by appending an opacity/alpha channel, and complementing it with several other measurable cues (texture/reflectance, motion/multi-view/BRDF sampling, compressive sensing, and polar/temporal modulation). Special attention is paid to implementation within a ring or spiral buffering architecture where frames are continuously ingested and periodically evicted.

Objectives

- Minimize the nullspace of the overall sensing operator and reduce metamers as far as possible given a laptop camera.
- Harvest robust spectral■temporal features from video frames (per■pixel / region) for later classification or segmentation.
- Do real■time or near■real■time processing using GPU shader passes (Three.js / WebGL / GLSL) where possible.
- Provide a canonical, testable pipeline that can be extended to learn mapping functions or accept additional sensors.

Selected Approaches (context: laptop webcam + software-only upgrades)

Below are the five approaches chosen for canonical specification. Each approach includes: motivation, concrete implementation steps, data/feature outputs, GPU/CPU notes, and evaluation suggestions.

1. Tetrachromacy Sensor (Append Alpha / Matte / Depth Proxy)

Motivation - Treat alpha/opacity or a computed matte/depth proxy as a fourth independent measurement channel (RGBA), expanding measurements from 3→4 and reducing the sensing nullspace. Implementation steps 1. Obtain alpha: source could be rendered alpha (if available), background-subtraction trimap matting, or a learned fast matting network operating on the ring buffer. If depth is available (webcam with depth or monocular depth estimator), compute a depth-to-opacity proxy. 2. Append alpha as a 4th channel in the shader pipeline: input vector = [R_lin, G_lin, B_lin, alpha]. Optionally learn a 4×4 projection M that maps input → sensor space. 3. Produce per-pixel 4■channel sensor outputs and store to an RGBA32F render target each frame. 4. Maintain temporal statistics (EMA mean, variance, min, max) per channel in ping-pong RTs. Data / features harvested - Per-frame (4-channel) sensor vector per pixel - Temporal mean & variance per sensor channel - Derived features: opponent dims, chromaticity, alpha statistics, alpha-normalized chroma GPU / CPU notes - All per-pixel ops (linearize, multiply, EMA) are cheap in GLSL and should run in real time on modern laptops. - Matting networks or monocular depth models run on CPU/GPU; for speed use lightweight models or WebGL-based matting implementations.

2. Exploit Reflectance, Texture and Shape Priors

Motivation - Spatial structure (texture, microstructure), BRDF cues and shape priors provide independent information to disambiguate materials that are spectrally similar. Implementation steps 1. Compute local texture descriptors per pixel/patch (local std, Laplacian energy, Gabor filters, local histograms). 2. Estimate simple BRDF proxies: specular/diffuse separation via chromaticity, specular highlight detection, roughness proxy via high-frequency energy. 3. Obtain depth/normal estimates using monocular depth models or structure-from-motion over the buffer; compute curvature metrics and thickness proxies. 4. Combine texture/shape features with color/alpha sensor vectors into a joint descriptor per pixel/region. Data / features harvested - Local texture metrics (std, energy, Gabor outputs) - BRDF proxies (specular strength, roughness) - Depth/normal/curvature maps or alpha-from-depth proxies - Aggregated patch descriptors for clustering/training GPU / CPU notes - Texture filters and Laplacian can be implemented as shader convolutions. Local histogram approximations can be computed via small neighborhood reductions or separable passes. - Depth and normals likely require a lightweight neural model on CPU/GPU; consider offline precomputation for testing.

3. Motion / Multi-view / BRDF Sampling

Motivation - Angular and temporal variation in reflectance gives extra independent constraints on surface reflectance and geometry; moving the camera or exploiting natural motion creates multi-view BRDF sampling. Implementation steps 1. Maintain a ring (or spiral) buffer of recent frames (N frames). Use optical flow to warp previous frames into current view for per-object aggregation. 2. Capture frames at slightly different viewpoints (natural hand motion or deliberate small camera motion). If possible, vary illumination angle (screen bounce or mobile light) to produce angular diversity. 3. Compute angular/radiometric differences per pixel (specular shift, highlight movement) and aggregate as additional features. 4. Optionally perform photometric stereo-like solves on short bursts if the illuminant directions are known or controllable. Data / features harvested - Flow-aligned stacked responses (per-pixel vectors across time/angles) - Angular difference maps and temporal derivative signatures - BRDF estimates (diffuse albedo, specular lobe parameters) where solvable GPU / CPU notes - Optical flow can be implemented with shader-based or neural approximations; use lower-resolution flow for speed and upsample statistics. - Photometric stereo requires controlled illumination; otherwise compute angular proxies and use them as discriminative cues.

4. Compressive Sensing & Basis Priors

Motivation - Natural reflectance spectra often lie in a low-dimensional smooth subspace. Use compressive sensing / basis priors to reduce unknowns and enable spectral recovery from limited measurements. Implementation steps 1. Build or adopt a spectral basis (PCA or learned dictionary) from public reflectance datasets or targeted sample captures. 2. Represent each spectrum $s(\lambda) \approx B * a$ where B is (N×K) basis and a is K coefficients (K ■ N). 3. Solve a constrained inversion per pixel from measurements $r \approx M * (B * a)$ using NNLS or regularized least squares (non-negativity and smoothness priors). 4. Optionally learn mapping from stacked measurements → coefficient vector a with a small neural network (supervised with hyperspectral data or screen-illuminant captures). Data / features harvested - Estimated basis coefficients per pixel (compact spectral embedding) - Reconstructed approximated spectra per region (for analysis) - Reconstruction confidence metrics and residuals GPU / CPU notes - Solving NNLS per pixel on GPU is feasible with iterative methods; for speed do coarse resolution or sparse sampling. - Learning-based inversion (small MLP) deployed in shader or on CPU is often the fastest practical route.

5. Polarization and Temporal Modulation

Motivation - Polarization discriminates specular vs subsurface scattering; temporal modulation (screen-driven illumination sequences) produces additional independent measurements without specialized cameras. Implementation steps 1. Polarization: if a linear polarizer is available, capture 2-4 frames with rotated polarizer orientations and record polarized responses. Use difference/ratio metrics to infer specular components. 2. Temporal modulation: use the laptop screen or a hand-held LED to display short sequences of distinct spectral content (red, green, blue, narrowband approximations). Capture synchronized frames to build a stack of measurements. 3. Align frames via flow, compute temporal-stack features, and optionally solve small inverse problems or train discriminative models on stacked responses. Data / features harvested - Polarization contrast maps (per-pixel specular/subsurface cues) - Multi-illuminant stacked responses (per-pixel vectors across modulation states) - Temporal response statistics and synchronization timestamps GPU / CPU notes - Polarized capture may require manual rotation or a rotating polarizer mount; simple two-angle capture (0°/90°) already provides useful cues. - Temporal modulation capture is easy to implement in browser: display fullscreen colors, capture frames, and store them into buffer. Ensure exposure and frame timing are matched.

Ring / Spiral Buffer Design and Usage

Buffer geometry and eviction policy - Use a ring buffer of size N (typical N = 8..32 depending on memory). Each new frame overwrites the oldest frame (FIFO). For spatially-adaptive retention, consider a spiral buffer that retains frames more often near recently-active regions (spatial weighting). Data stored per buffer slot - RGBA32F sensor RT (4 channels per pixel) for the frame - Flow field (coarse resolution) to align previous frames - Timestamps and exposure/illumination metadata - Optional compressed patch descriptors for quick lookups Alignment & aggregation - Before aggregation, warp previous frames' sensor RTs using optical flow to the current view. Use EMA with $\alpha \approx 2/(N+1)$ to approximate N-frame window statistics. - For moving objects, maintain per-track buffers (tracked by centroid or keypoint) to avoid mixing different surfaces due to occlusion. Memory considerations - RGBA32F at 640×480 per frame $\approx 640*480*4*4$ bytes ≈ 4.9 MB. With N = 16, memory ≈ 78 MB (plus flow & RTs). For larger frames or longer buffers, downsample or store lower-precision (half floats).

Feature Schema — What we harvest per pixel / region

Per-pixel feature vector (recommended): - t0: instantaneous sensors: [s1, s2, s3, s4] (4 floats) - Temporal stats: mean_1..4, var_1..4, min_1..4, max_1..4 (16 floats) - Derived: opponent dims (2 floats), chromaticities (2 floats) - Motion: flow magnitude (1 float), Doppler-like delta (1 float) - Texture: local std, Laplacian energy (2 floats) Total per-pixel ~ 28 floats (example). For storage, compress per-region or quantize. Storage formats - GPU: RGBA32F RTs for sensors, mean, variance; additional RGBA32F RTs for texture/flow proxies. - Offline: HDF5 or numpy arrays with per-frame metadata; or columnar parquet/CSV for patch-level data.

Evaluation Plan & Experiments

Suggested experiments: 1. Baseline: implement RGBA tetrachrome pipeline with EMA and visualize opponent channels + variance. Measure clustering of labeled patches (skin, cloth, plastic). 2. Active illumination: run screen modulated capture (6 colors) and test discriminative gains vs baseline. 3. Polarization ablation: compare two-angle polarized capture vs none on specular-heavy materials. 4. Compressive sensing: build spectral basis and test NNLS reconstruction on synthetic spectra; quantify reduction in metamers. 5. Motion/BRDF: collect multi-view bursts via small handheld motion and evaluate BRDF proxies' separability power. Metrics: - Per-class F1, accuracy on labeled patches - Silhouette score and inter-cluster distances in sensor feature space - Fraction of synthetic metamers separated (if spectral data available) - Memory/throughput: frames per second, GPU/CPU usage

Practical Considerations & Pitfalls

- Motion alignment is critical for temporal stacking; poor flow causes mixing and reduces gains. - Exposure/saturation: avoid clipping; use multi-exposure capture to mitigate. - Domain shift: learned priors must include lighting/camera variability. - User safety/comfort: avoid high-frequency flicker or intense flashes. - Memory: manage buffer size and downsampling to fit target hardware.

Implementation Checklist (priority-ordered)

Step	Task
1	Implement RGBA sensor shader + RGBA32F RT write
2	Implement EMA & variance ping-pong RTs (N=16 default)
3	Integrate alpha estimation (fast matting or bg subtraction)
4	Add optical-flow warping for frame alignment (coarse)
5	Implement local texture filters (shader) and depth proxy pipeline
6	Create active-illumination capture page for screen modulation
7	Collect labeled patches and run LDA/metric-learning to produce projection M
8	Evaluate metrics and iterate

Conclusion

Combining a tetrachromacy-like sensor (alpha as 4th channel) with texture/shape priors, motion/BRDF sampling, compressive sensing priors, and polarization/temporal modulation provides a powerful, practical toolkit to reduce metamers and improve material/shape discrimination using only a laptop camera plus software-controlled illumination. This document is intended as a canonical reference for implementing and testing these methods within a ring/spiral buffer architecture. The next step is to implement the prioritized checklist and run the evaluation experiments described above.